

CsoundUnity 4.0: Toward a Csound-Based Interactive Audio Platform for Unity

Giovanni Bedetti

Independent Researcher, Urbino, Italy
write@giovannibedetti.com

Abstract. CsoundUnity is an open-source Unity package embedding the Csound audio engine, exposing its synthesis and signal-processing capabilities through a `MonoBehaviour` component and its custom Inspector. This paper presents version 4.0's changes, marking a shift from a channel-driven wrapper toward an integrated, interactive audio environment.

Version 4.0 upgrades the embedded engine to Csound 7 (beta) across all platforms (macOS, Windows, Android, iOS, visionOS, WebGL) and adds new functionality in several areas. Native low-latency input backends (CoreAudio, WASAPI, AAudio, RemoteIO) bypass Unity's buffered Microphone API through a native callback it does not provide; the AAudio backend reports approximately 4 ms of input latency on a Pixel 4a. An audio-routing layer lets CsoundUnity instances feed one another's input, with cycle detection and an interactive node-graph editor; two new `AudioPath` options, `IAudioGenerator` and `RootOutput` (Unity 6.3+), join the existing `OnAudioFilterRead` callback – the former feeds an `AudioSource` directly, the latter writes straight to Unity's main output, bypassing both `AudioSource` and mixer. Timeline integration adds channel-automation clips and eight score-generation clip modes, spanning hand-authored Score lines, an arpeggiator and chord generator, two step-sequencer variants, and generative Euclidean, stochastic, and swarm-based event generation.

Together with a UI widget system auto-generating a playable interface from a Csound document's Cabbage GUI layout, cross-platform MIDI input, and a new vector-morphing layer built on the existing preset system, these additions move CsoundUnity toward an integrated, multi-platform environment for performing Csound instruments inside Unity – the same project is portable across desktop, mobile, and web. We close with directions for future work, including native audio output.

Keywords: Csound, Unity, game audio, real-time audio, algorithmic composition

1 Introduction

CsoundUnity is an open-source Unity package that embeds the Csound audio engine [1] inside Unity's component-based architecture, wrapping it in a `MonoBehaviour` component attached to any `GameObject`. It bridges two audiences: Csound users gain Unity's real-time, cross-platform runtime – games, XR, interactive installations – while Unity developers gain Csound's text-based synthesis and signal-processing language, orchestras and instrument libraries built over four decades. The package has been maintained since 2015 [2] and has gone through several major revisions, historically exposing Csound mainly as a channel-driven black box: instantiate Csound, start/stop runtime-compiled instruments, read/write named channels on Csound's software bus.

Version 4.0 marks a shift away from that minimal surface, driven by four recurring needs: latency low enough for live performance, a way to route audio between instances, a way to sequence note-level events – rhythms, pitches, patterns – without leaving Unity, and a way to expose an instrument's controls as a playable on-screen UI for end users – one that ships in a build, where channels are otherwise reachable only from C# code. This paper documents the resulting additions.

2 Csound 7 Migration

Version 4.0 upgrades the embedded engine from Csound 6 to Csound 7 (beta) across every supported platform – macOS, Windows, Android, iOS, visionOS, and WebGL. Csound 7 streamlines the host API [3]; CsoundUnity's C# binding layer (`CsoundCsharp.cs`, `CsoundUnityBridge.cs`) was updated accordingly.

These are binding-layer changes: a Csound document (CSD) authored for CsoundUnity 3.x remains source-compatible with 4.0, barring the few opcodes Csound 7 itself renames or removes.

3 New Functionality in Version 4.0

3.1 Low-Latency Native Audio Input

CsoundUnity’s default audio path runs on Unity’s own audio thread, whose configurable DSP buffer (512–1024 samples, roughly 11–23 ms at 44.1 kHz) is too coarse for responsive instrument playing or live microphone processing. Version 4.0 adds NativeAudioInput, a family of platform-specific plugins built from a companion repository, capturing audio outside Unity’s audio graph: CoreAudio/AUHAL (macOS), WASAPI (Windows, shared/exclusive), AAudio (Android, MMAP low-latency path, exclusive when available), and AudioUnit RemoteIO (iOS/visionOS).

Each backend follows the same architecture: the platform’s audio callback – running on its own real-time thread, outside Unity entirely – writes captured samples into a lock-free, single-producer/single-consumer ring buffer; Unity’s audio thread then drains one `ksmps` block (one Csound control cycle) from that buffer before each `PerformKsmps` call, writing it into Csound’s input (spin) buffer, where the running instrument reads it via `inch`. Csound 7 removes the per-sample spin setter in favor of the raw `csoundGetSpin` pointer [3] (cached once it becomes valid, after the first `PerformKsmps` call), so the consumer writes directly into that buffer:

```
// device callback thread (producer)
void OnDeviceCapture(const float* samples, int n) {
    cni_rb_write(ringBuffer, samples, n);
}

// Unity audio thread, before each PerformKsmps (consumer)
// spinPtr cached once via csoundGetSpin(), after the first PerformKsmps
void FillSpinBuffer(MYFLT* spinPtr, int ksmps, int nchnlsIn) {
    cni_rb_read(ringBuffer, tmp, ksmps * nchnlsIn);
    for (int i = 0; i < ksmps * nchnlsIn; i++)
        spinPtr[i] += tmp[i]; // additive: audio routes feed spin too
}
```

Simplified producer/consumer flow between the native capture callback and Csound’s spin buffer.

Once the buffers are sized at startup, neither side locks or allocates. On Android the AAudio backend reports approximately 4 ms of input latency at 48 kHz on a Pixel 4a; all four backends also expose multichannel capture through a native callback – unlike Unity’s `AudioClip`-based `Microphone` API. For this low-latency path specifically, Unity’s Enhanced Audio Foundation is expected to gain device enumeration and multi-device I/O later, but ships neither as of Unity 6.3 [4].

3.2 Audio Routing between Instances

Multiple CsoundUnity instances in the same scene previously had no way to exchange audio. Version 4.0 adds an audio-routing layer: an instance’s spin buffer can be fed directly by a named output channel of another instance in the scene, chaining one instance’s processing into another’s input. To avoid infinite feedback loops, the routing layer rejects any new route that would create a cycle in the routing graph, unless explicitly forced. Because routing graphs are hard to keep straight from code alone, an interactive node-graph editor (`AudioRouteGraphWindow`) visualizes and edits the whole route graph across every CsoundUnity instance in the scene (Fig. 1).

3.3 New Unity Audio Paths

To keep pace with Unity 6.3’s new interface-based scriptable audio pipeline (unavailable in Web builds), version 4.0 adds two `AudioPath` options alongside the pre-existing `OnAudioFilterRead` [5] callback. `IAudioGenerator` [6] attaches to an `AudioSource` as its generator instead of passing through `OnAudioFilterRead`’s filter chain, avoiding the garbage-collection hiccups of the managed callback [4] while keeping full `AudioMixer` routing and 3D spatialization. `RootOutput` [7] needs no `AudioSource`, writing samples directly into Unity’s main audio output and bypassing the `AudioMixer` for the shortest signal path of the three – at the cost of 3D spatialization, mixer effects, and built-in per-instance gain staging.

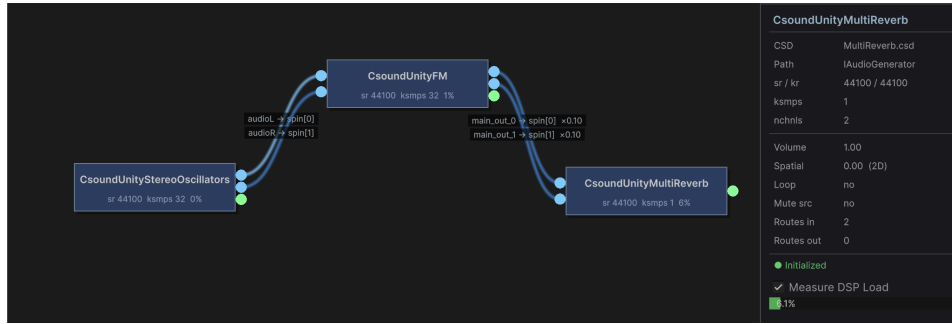


Fig. 1. The `AudioRouteGraphWindow`: three `CsoundUnity` instances chained via audio-routing edges, with per-instance details (CSD, `AudioPath`, sample/control rate, route count) shown for the selected node.

3.4 Timeline-Based Composition Tools

Version 4.0 integrates Csound composition and performance directly into Unity’s Timeline window [8]. Two clip types are available: channel-automation clips, ramping a named Csound channel between values (fixed, random, or randomized-and-smoothed), and score-generation clips, which produce Csound score events. The latter cover eight distinct modes: a Score clip for hand-authored score lines; an arpeggiator and a chord generator; two step-sequencer variants – a drum-machine-style Pattern clip (Boolean per-step grid, independently assignable lanes) and a Step clip (per-step pitch, velocity, duration); and three generative algorithms – Euclidean rhythms [9], stochastic triggering, and swarm-based event generation, the last producing dense, granular-like textures. In a real-time, interactive setting, the stochastic and swarm modes recall Tenney’s [10] Bell Labs stochastic works, while the rule-based generative modes echo Koenig’s [11] *Project 1* and *Project 2*.

Timing-sensitive modes (arpeggiator, Euclidean, Pattern, Step) offer two scheduling strategies. In *Precise* mode, tempo and discrete parameters are snapshotted once per pattern cycle, giving internally consistent timing at the cost of tempo changes only taking effect at the next cycle boundary. In *per-note/per-step* mode, tempo is re-read at every note or step, so live tempo changes are audible immediately – suited to performance over rhythmic regularity. Both dispatch through the same mechanism: Unity evaluates the Timeline once per frame, but each note reaches Csound as an `i`-statement sent via `SendScoreEvent`, its onset (`p2`) precomputed to correct for the Timeline’s per-frame evaluation drift rather than left to direct frame-rate quantization. A companion library, `MusicUtils`, supplies the underlying music-theory primitives – scales, chords, arpeggio patterns, Euclidean generation – shared across clip types.

3.5 UI Widgets, Inspector Improvements, and New Samples

CsoundUnity has always driven its custom Inspector from the CSD, parsing the `<Cabbage>` [12] section into channel controllers – one per widget – each exposed as an Inspector control, its channel(s) accessible via `GetChannel/SetChannel`. Version 4.0 extends the recognized widgets (encoder, XY pad, range sliders, form-level attributes) and adds validation: refreshing a CSD spins up an isolated Csound instance on a background thread (`CsoundWorker`) that compiles it, runs one control cycle, and queries the allocated channel list – catching compile errors and missing channels without entering Play mode.

On top of this data, version 4.0 adds matching Unity UI components – sliders, knobs, an endless rotary encoder, buttons, toggles, a dropdown, an XY pad, a virtual keyboard, meters, range sliders – each bound to a Csound channel by name. Unlike the Editor-only Inspector, these run in a build, giving the end user a playable interface. A *Create UI* button auto-generates a full Unity Canvas from the parsed Cabbage layout, using a `CsoundUnityUISettings` asset to map each widget type to a prefab (a reusable template); *Update UI* re-syncs the hierarchy after CSD changes. Both `CsoundUnitySlider` and the Inspector’s own channel sliders reproduce Cabbage’s `range(min, max, val, skew, increment)` syntax: skew reshapes the 0–1 slider position through a power curve (linear at skew = 1, non-linear otherwise), and increment quantizes it to discrete steps.

The Inspector also gains a Play-mode audio monitor – waveform, spectrum, oscilloscope, Lissajous, and spectrogram views, each zoomable – reading directly from Csound’s output buffer for debugging and sound design without leaving the Editor.

Version 4.0 also bundles many new samples, among them *Engines*, with a CSD by Jeanette C., synthesizing engine audio from vehicle speed and switching presets at runtime, and *Bouncing Ball*, porting Janevska, Anderson, and Heintz’s Csound implementation [13] of Andy Farnell’s bouncing-ball model. Two historic Csound works join them: Iain McCurdy’s generative *Csound Haiku*, and Richard Boulanger’s *Trapped in Convert*, restaged as an interactive scene where each dropped instrument claims unique named audio channels at runtime and plays through its own spatialized `AudioSource`. Combined with the Timeline’s Pattern/Step sequencers and an auto-generated Cabbage UI, these samples let an artist start with rhythm and interface covered, ready to develop generative sound in Unity using Csound.

3.6 Cross-Platform MIDI, WebGL Continuity, and Presets

Version 4.0 also adds `CsoundUnityMidiInput`: CoreMIDI (macOS/iOS/visionOS), `android.media.midi`, WinMM (Windows; short messages only), and Web MIDI (WebGL). WebGL support dates to version 3.5; 4.0 continues it under the same limits – a two-channel microphone cap common to browser `getUserMedia` implementations [14], Web MIDI requiring HTTPS and a supporting browser.

A vector-morphing layer, `CsoundUnityVectorMorph`, sits on top of the existing preset system: it bilinearly blends the channel values of four `CsoundUnityPreset` assets across a 2D pad via an interactive editor, letting one control surface interpolate between four saved instrument states; blending covers the primary (bottom-left) preset’s channels, falling back to its value where the others lack one.

4 Future Work

Native audio output – a handle-based backend mirroring `NativeAudioInput`’s coverage, paced by a semaphore signalled by the hardware callback rather than by a fixed timer – is under active development within the 4.0 release cycle. Beyond it, further directions include `CsoundUnityMidiOutput` to pick a MIDI output device at runtime (the physical device is fixed via Csound’s `-Q` flag at engine start; `midout`’s `iport` argument only selects among already-open ports), a handle-based multi-instance refactor of `NativeAudioInput`, and Linux support.

5 Conclusion

Version 4.0 moves `CsoundUnity` beyond a channel-driven wrapper, toward an environment with its own native low-latency audio input, inter-instance routing, timeline-based composition, and a Cabbage-derived widget system – while retaining Unity’s cross-platform deployment.

Availability. `CsoundUnity` 4.0 is open-source at github.com/rorywalsh/CsoundUnity.

Acknowledgements. The author thanks Rory Walsh for creating `CsoundUnity`, Richard Henninger for the original .NET Csound interface, and Iain McCurdy, Jeanette C., Richard Boulanger, and Janevska, Anderson, and Heintz for the bundled sample instruments.

References

1. Lazzarini, V., Yi, S., ffitich, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Springer (2016)
2. Walsh, R.: Csound and Unity3D. In: Proceedings of the 3rd International Csound Conference (ICSC 2015), St. Petersburg (2015)
3. Csound Developers: Csound Host API Migration Guide: Csound 6 to 7, github.com/csound
4. Unity Technologies: Audio Status Update Q3 2025, Unity Discussions, discussions.unity.com
5. Unity Technologies: MonoBehaviour.OnAudioFilterRead, Unity Scripting API, docs.unity3d.com
6. Unity Technologies: IAudioGenerator, Unity Scripting API, docs.unity3d.com
7. Unity Technologies: Example: Create a root output, Unity Manual, docs.unity3d.com
8. Unity Technologies: Timeline, Unity Manual, docs.unity3d.com
9. Toussaint, G.T.: The Euclidean Algorithm Generates Traditional Musical Rhythms. In: Proceedings of BRIDGES: Mathematical Connections in Art, Music and Science, pp. 47–56. Banff (2005)

10. Tenney, J.: *From Scratch: Writings in Music Theory*. Polansky, L., Pratt, L., Wannamaker, R., Winter, M. (eds.). University of Illinois Press (2015)
11. Koenig, G.M.: *Composition Processes*. In: *Computer Music: Reports on an International Project*. Canadian Commission for UNESCO, Ottawa (1980)
12. Walsh, R.: Cabbage, a new GUI framework for Csound. In: *Proceedings of the Linux Audio Conference*, Cologne (2008)
13. Janevska, M., Anderson, J., Heintz, J.: Implementing Andy Farnell’s ‘bouncing ball’ in Csound. In: *Proceedings of the 6th International Csound Conference (ICSC 2022)*, Athlone (2022)
14. Mozilla: channelCount constraint for microphone input, *Advancing WebRTC*, blog.mozilla.org. All URLs last accessed July 2026